Deterministic Gates over Inference-Time Scaling: A Compute-Time-Aware Harness for Consistent In-Car Assistants

Dongjun Jeong
Narnia Labs
june.jeong@narnia.ai

Abstract

CAR-bench evaluates in-car assistant agents on task success, on consistency (Pass^3: success in all three independent trials), and on limit-awareness. Our base model, Cerebras-hosted `gpt-oss-120b`, is capable but inconsistent: it solves most tasks at least once (high Pass@3) yet rarely three times in a row (low Pass^3). We argue that Pass^3 is bought by determinism, not by inference-time scaling, and that determinism also lowers latency, which matters for a fast-reasoning track. Our harness moves every decision it can from variable, latency-incurring LLM calls into spec-derived deterministic gates with zero LLM calls and zero variance, and reserves the model for the open-ended semantic layer that only it can handle. A three-way ablation supports the thesis: self-consistency, response grounding, and best-of-N all fail to improve Pass^3, two of them hurt it, and each adds latency. The harness lifts `gpt-oss-120b` from a bare-model score of 0.28 to about 0.50 (average Pass^3 across the three task types), competitive with frontier proprietary models on this benchmark, using the weakest open model in the field and well under the token budget.

1 Introduction

CAR-bench [Kirmayr *et al.*, 2026] is a benchmark for in-car AI assistants. A driver asks for something in natural language, and the agent carries it out over a multi-turn dialogue by calling vehicle, navigation, charging, and productivity tools. Modern LLMs handle such requests well on average, but they remain probabilistic generators: the same request, asked three times, can succeed twice and fail once. For an assistant that controls a car, this is the wrong shape of reliability. Getting the intended action right must not be a matter of one good attempt among many; it has to happen consistently, every time the driver asks. An assistant that opens the sunroof correctly on only one of three identical requests is unsafe. CAR-bench operationalizes this with the Pass^k metric (consistency), which grants task credit only if the task is solved in all k trials, alongside Pass@k (potential), which requires success in at least one. A large gap between the two reveals latent competence blocked by inconsistency.

Track 2 constrains the reasoning model to Cerebras `gpt-oss-120b` [OpenAI, 2025] and is judged on two axes: a Pass^3 rank award and a Cerebras award for compute-time-aware, faster-inference design. This paper describes the harness we submitted and the experiments behind its design choices. Our central claim covers both award axes: the design choices that raise Pass^3 are the same ones that lower latency, because both are maximized by (a) replacing variable, slow LLM decisions with deterministic zero-latency code and (b) minimizing the number of LLM decision points, each of which is a source of both variance and delay. Inference-time scaling adds calls [Wang *et al.*, 2023] and therefore moves in the wrong direction on both axes. Our experiments confirm this.

2 Background

Task. The agent holds a multi-turn dialogue with an LLM-simulated user, calls the 57 interconnected tools exposed in the challenge environment (get tools for information retrieval, set tools for environment modification) across six domains spanning navigation, productivity, charging, and vehicle control, and must obey 19 domain-specific policies, 12 verified via code-based checks and 7 via LLM-as-a-Judge. The setting extends the tool-agent-user design of τ-bench [Yao *et al.*, 2025; Yao *et al.*, 2023]. The CAR-bench error taxonomy distinguishes premature actions (E1, executing before gathering necessary context), policy violations (E2), logical errors (E3), execution errors (E4), and fabrication (E5, implicit or active); inconsistency across trials is the failure that Pass^3 isolates.

Base model behavior. On the public test split, bare `gpt-oss-120b` averages Pass^3 0.28. Even the repo's planner-template baseline scores Pass^1 0.49 but Pass^3 0.32, and on Disambiguation its Pass@3 of 0.68 collapses to Pass^3 0.20, the consistency gap that CAR-bench isolates. The model has the capability but does not apply it consistently.

3 Method: A Determinism-First Harness

We build on the planner/executor template provided in the challenge starter repo: a private planner produces a reusable plan, and an executor emits one benchmark-visible action per

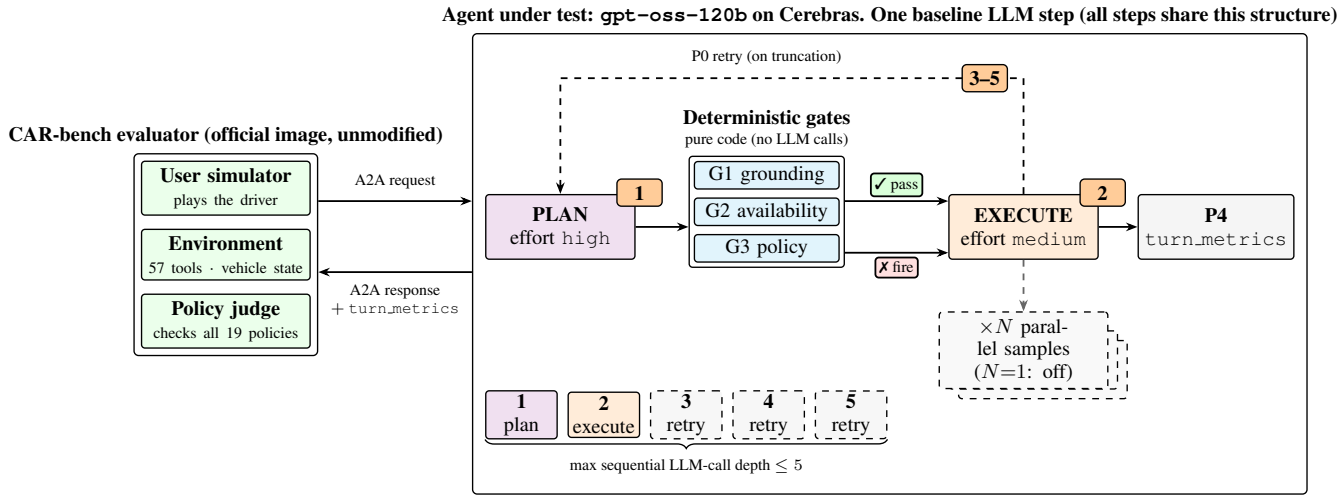

Figure 1: Agent architecture for one baseline LLM step, documented per the Track 2 audit requirement. The colored solid blocks are the only LLM calls (violet: PLAN, orange: EXECUTE), and the slot ledger (bottom) matches their colors: it gives the maximum sequential LLM-call depth of 5, asserted in code. Calls ① plan and ② execute always run; the gray dashed slots ③–⑤ are reserved for the P0 retry loop (dashed back-edge), which re-invokes the failing stage only on `finish_reason=length`, with a doubled budget and one step lower reasoning effort per iteration. Branches at the gate layer (checked box: pass; crossed box: fire, a corrective directive to the executor) add no LLM calls. Parallel fan-out (stacked cards: self-consistency, best-of-N lenses, grounding pass) is exempt from the sequential limit and disabled (N=1) in the submitted configuration. The agent ships as a digest-pinned public GHCR image; the evaluator (left) is the official, unmodified organizer image. Token usage is reported only through the existing A2A `turn_metrics` fields (`prompt_tokens`, `completion_tokens`, `thinking_tokens`); no custom sequential-depth metadata is added.

turn. `gpt-oss` exposes low, medium, and high reasoning-effort levels; we run the planner at `high`, where deep reasoning stays private, and the executor at `medium`. Around this we add three deterministic gates and two robustness fixes, P0 and P4 (internal work-package identifiers, retained for traceability to the public development log; the gates were developed as P1a, P1b, and P2 in that log) (Figure 1). All gates run before execution and check only what the agent can know a priori: value grounding, policy preconditions, tool availability. They never compare an action against expected outcomes, which the rules forbid.

Gate 1: preference-first (E1). Before a set tool call whose value the user did not state, the gate forces a `get_user_preferences` lookup (plus relevant state); the deferred action re-emerges next turn with context in hand. A question gate likewise defers the first clarification question until preferences are checked. The rationale is spec, not data: the benchmark's disambiguation protocol ranks preferences and context above asking the user. Extra reads are free under the tool subset metric, which requires all ground-truth get tools to be invoked while allowing additional ones.

Gate 2: availability (E5). A Hallucination task removes a required tool, tool parameter, or tool result, and calling a removed element fails instantly. The gate deterministically detects (i) unavailable tools, (ii) schema-unknown or missing-required parameters, (iii) enum-invalid values, and (iv) tool results containing the literal `"unknown"`, steering the model to acknowledge the missing capability instead of fabricating.

Gate 3: policy preconditions (E2). Derived one-to-one from the 19 wiki policies: required prior lookups (for example, weather in a separate earlier step before opening the sunroof, per the evaluator's `previous_tool_calls` check), one navigation edit per step, phone-call last, and a current-day guard for calendar and weather. Policies verified via LLM-as-a-Judge (metric units, toll and route disclosure, single-zone temperature notice) are enforced by explicit executor rules plus a deterministic 12h-to-24h time normalizer.

P0: length repair. `gpt-oss` spends its output tokens on reasoning before the answer, so at high effort a low output cap cuts the plan JSON off mid-generation (`finish_reason=length`). We raise the per-call output-token limit (`max_completion_tokens`) to 8192 for the planner and 4096 for the executor; if truncation still occurs, the failing stage retries with a doubled limit and one step lower reasoning effort, and a raw error is never shown to the user.

P4: budget accounting. The competition limit is an average of 500K tokens per task; our mean is 155K. We therefore never fail a task for token reasons, since a per-task cap would wrongly kill a task that needs the tokens. Token thresholds are monitor-only by default. P0 and P4 are robustness and compliance fixes rather than scoring levers; their effect appears in the generation-level progression (Table 1) and in the per-run records of the development log, not as isolated ablations.

All rules are spec-derived, from the public wiki policies and tool schemas that every agent receives, so they generalize to the whole task distribution rather than fitting observed instances (see Section 6).

3.1 Compliance and Compute Assumptions

Track 2 budgets. The agent is a dockerized A2A-compatible image using direct Cerebras-hosted `gpt-oss` inference. Per baseline LLM step it issues two sequential LLM calls (plan, execute) plus bounded length-repair retries, asserted in code against the five-sequential-call limit (Figure 1). Branches at the gate layer select the executor's directive and add no LLM calls, and the retry loop fires only on truncation, so the worst-case sequential depth per baseline step is five. Parallel fan-out, which is exempt from the sequential limit, is implemented with threaded per-sample clients but disabled (N=1) in the submitted configuration after the negative ablation (Section 5). Token usage, covering input, reasoning, and output, is reported per turn through the A2A `turn_metrics` token fields and logged for countercheck. The measured mean is 155K tokens per task against the 500K-average limit, so token thresholds are monitor-only and never alter or fail a task.

No evaluator exploitation. Gates check only a-priori preconditions (schemas, policy prerequisites, value provenance) available to any agent. The harness never self-evaluates against task-level metrics or subscores, never repairs a response against scoring criteria, and encodes no test-set answers or task-specific lookup tables. The single instance-derived heuristic that emerged during development, a keyword list, was removed before submission (Table 1, v2).

Rate-limit assumptions. Development assumed the elevated Cerebras limits granted to Track 2 (observed: 1M tokens/min), which the agent never approached. The practical development bottleneck was the evaluator-side Gemini user-simulator quota, not agent inference. All model names, provider routes, reasoning-effort selectors, and harness features are exposed as environment variables, so each ablation was a configuration change rather than a code change. The full harness, scenario configs, and development logs are publicly available under the MIT license.[1]

4 The Compute-Time-Aware Thesis

Pass^3 rewards consistency; latency penalizes redundant inference. Both are maximized by the same two moves. First, move decisions from LLM calls to code: a gate decision is perfectly consistent and costs no latency. Second, minimize the number of LLM decision points: `gpt-oss` is not bit-deterministic even at temperature 0, so each additional call adds both variance and delay. Inference-time scaling adds calls and worsens both axes. The reason is simple arithmetic: a task with per-trial success probability p has expected Pass^3 of p^3, so probabilistic gains are cubically discounted, while a gate pins $p \approx 1$ for its failure class outright.

Reliability decisions (availability, preconditions, grounding of value provenance) are made by gates, with no LLM calls and no variance. The model handles only the irreducibly semantic layer: intent understanding, natural-language disambiguation, free-text responses. There the harness stabilizes the model with temperature 0, structured output, and fixed tie-breaks. Width runs as threads with per-sample clients, so N samples take the wall-clock time of one; Section 5 shows that width does not pay for Pass^3 here.

[1] https://github.com/dongjuns/car-bench-ijcai

Config	Base↑	Hall.↑	Dis.↑	SCORE↑
bare `gpt-oss-120b`	.36	.37	.12	.280
+ planner template	.44	.32	.20	.320
+ deterministic gates (v1)	**.54**	.38	.36	.427
+ spec coverage (v2)	.52	**.50**	.44	.487
+ prompts (v3, submitted)	**.54**	.42	**.52**	**.493**

Table 1: Pass^3 by task type and SCORE (unweighted average across task types) on the public test split (3 trials). Rows are cumulative. The bare row is from [Kirmayr *et al.*, 2026]; the planner template is the starter-repo baseline; v2 widens the gates to the full policy and schema spec and removes an instance-fit keyword heuristic; v3 adds prompt reinforcements. Bold marks the best value in each column.

5 Experiments

5.1 Evaluation Setup

We evaluate the harness on the public test split of CAR-bench under the official protocol: 3 trials per task, with SCORE defined as the average of Pass^3 across the three task types, not weighted by tasks. The user is simulated by Gemini-2.5-Flash (thinking), and policy compliance is checked via code-based checks and LLM-as-a-Judge. We report successive generations of the harness and an ablation of inference-time scaling. Mean measured LLM latency is 10.4 to 10.9 s per run at 155K mean tokens per task.

5.2 Main Results

Table 1 shows the SCORE progression. The harness lifts the weakest open model (bare .28, last among non-degenerate models on the CAR-bench leaderboard) to about 0.50, competitive with Claude-Sonnet-4 (thinking) at .47 and above GPT-4.1 (.37) and Gemini-2.5-pro (auto-thinking) at .38 on this benchmark. A transient evaluator-side user-simulator error failed two Base tasks independently of the agent; correcting for those two runs puts the SCORE at about .50 to .51. The Hallucination dip from v2 (.50) to v3 (.42) is within sampling noise at n=50 (SE $\approx$ 7pp): none of the v2-to-v3 changes targeted Hallucination, and v3 reaches its highest Hallucination Pass@3 (.80), so the two configurations are statistically indistinguishable on Pass^3 and v3 was selected on SCORE. The mean is 155K tokens per task, well under the 500K limit.

5.3 Ablation: Inference-Time Scaling

We tested three ways to spend more inference (Table 2). Self-consistency samples the model at temperature > 0 and takes a majority vote. The grounding pass re-checks a drafted response against the tool results with one extra LLM call. Best-of-N drafts N candidate responses in parallel and selects one with a verifier pass. Self-consistency and grounding raise per-trial capability (Pass^1, Pass@k) yet reduce Pass^3 by injecting trial-to-trial variance, and best-of-N buys no significant Pass^3 gain at twice the latency. Each lever adds compute; none improves the ranked metric. The grounding

Method	Pass^3 without	Pass^3 with	extra compute
self-consistency	.44	.40	+42% tokens
response grounding	.50	.40	+1 LLM call per turn
best-of-N	.26	.29†	+65% tokens, 2× latency

Table 2: Ablation of inference-time scaling. Each row compares Pass^3 without and with the lever, on otherwise identical configurations, measured on the task type the lever targets: self-consistency on Disambiguation (test), grounding on Hallucination (test), best-of-N on Disambiguation (train). †Within sampling noise (n=31, SE ≈ 8pp). None of the levers improves Pass^3.

pass is the sharpest illustration of the generation–verification trade-off [Cobbe *et al.*, 2021]. It catches fabrications and lifts Hallucination Pass^1 from .56 to .76, yet drops Pass^3 from .50 to .40. The gated configuration was nearly deterministic ($.50 \gg .56^3$); the extra pass pushes it back toward the i.i.d. regime ($.40 \approx .76^3$). Scaling improves capability but not consistency, and it costs compute. Gates and temperature-0 decoding improve consistency at zero compute, which is the better trade for a fast-reasoning track.

5.4 Generalization

On the held-out train split, which we never optimized against (only static analysis was performed during development), the Disambiguation probe shows a consistency gap: train Pass^3 0.26 vs. test 0.44, with capability (Pass@3) similar on both sides (0.65 vs. about 0.60). At $n \approx 25$–31 per split (SE ≈ 8–10pp) the 18pp gap is about 1.4 combined standard errors. We do not dismiss the signal, but it is consistent with sample noise and split difficulty; the mechanism-level gates transfer by construction, and the only instance-fit heuristic, a keyword list, was removed before submission.

6 Discussion

Model ceiling. Residual failures are dominated by state-value reasoning (using looked-up values to produce correct set-actions) and by free-text fabrication, the open semantic layer. These cannot be gated without imitating the ground-truth-based reward, which is forbidden, and inference-time scaling does not fix them at Pass^3. This bounds the Pass^3 attainable by any harness on this model.

What fast inference on a weak model buys. Track 2 pairs a model that is weak relative to the frontier with inference that is much faster. The common way to spend that speed is inference-time scaling: sample more, vote, verify. Section 5 shows this fails under a consistency metric, because every added stochastic call converts speed into variance. Our results suggest the dividend is better spent on structure. At about 10.5 s of LLM latency per run, the harness affords a two-stage pipeline per turn (private planning at high reasoning effort, execution at medium) plus retries, a budget that would be impractical at frontier-model latencies for an interactive in-car assistant. Asymmetric reasoning-effort routing was the one compute knob that paid: deep reasoning runs once, privately, where its verbosity and variance are contained, and the benchmark-visible action comes from a cheaper, more constrained call. The fast-weak regime rewards spending latency on more deterministic structure per turn instead of more samples per decision.

Inference techniques compatible with Pass^3. Worth pursuing are techniques that add no trial-to-trial randomness. Grammar-constrained decoding is deterministic and removes format failures. Speculative decoding [Leviathan *et al.*, 2023] is output-identical to the target model, a pure latency win. Batch-invariant kernels would make temperature-0 decoding reproducible and turn Pass@1 into Pass^3 by construction; current serving stacks do not guarantee this, and the residual nondeterminism is a floor under any harness. Sampling methods can be salvaged only by making them deterministic end to end, and provider-side nondeterminism still remains.

Outlook: post-training for consistency. Track 2 fixes the model, so weight updates are out of scope here, but the residual failures above are the kind that post-training addresses better than harnessing. Two directions look promising to us. First, consistency-aware RL: standard verifiable-reward RL optimizes expected success and is indifferent between a policy that solves one task every time and one that solves three tasks a third of the time each; a Pass^k-style objective over grouped rollouts (in the spirit of group-baseline methods such as GRPO [Shao *et al.*, 2024]) would optimize the deployed metric directly. Second, targeted distillation of frontier trajectories on the failing slice, state-value reasoning in particular. A small model that has internalized the policy wiki needs fewer harness rules; the gates then act as a safety net rather than a substitute for model competence.

Development on the test split. We tuned against the public test split (reported here) and validated on train; conventional hygiene tunes on train and reports on test. We disclose this. The reported test numbers are mildly optimistic, and the train numbers are the clean held-out estimate. Because the harness is a low-capacity spec-derived rule set rather than a fitted high-capacity model, we expect the generalization gap to be small.

7 Conclusion

We set out to make a weak model behave consistently on CAR-bench, and the approach is simple. Every decision that can be derived from the public spec is checked by a gate, in code, before execution, at no LLM cost. The model handles only what code cannot: understanding intent, resolving ambiguity, writing responses. Decoding is pinned at temperature 0.

This works because of the metric's arithmetic. Pass^3 multiplies a per-trial success probability three times, so a probabilistic improvement is cubed away, while a code decision holds at $p \approx 1$. Our ablation confirms the corollary from the other side: self-consistency, grounding, and best-of-N each add stochastic calls, and none of them improves Pass^3.

The result supports the strategy. Bare `gpt-oss-120b` scores 0.28; with the harness it reaches about 0.50, near frontier models, at 155K tokens per task and about 10.5 s of LLM

latency per run. On fast-reasoning hardware, the compute-aware design was the one that spent less inference. What remains, state-value reasoning and free-text fabrication, is the model's own ceiling, and closing it is a job for post-training, not for more inference.

References

[Cobbe *et al.*, 2021] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, 2026. Association for Computational Linguistics.

[Leviathan *et al.*, 2023] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

[OpenAI, 2025] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

[Shao *et al.*, 2024] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[Yao *et al.*, 2025] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025.